



GROUP 28 · CSIT321

Sprint Progress Against the A2 Plan

Certacito.ai · A4 Prototype Presentation appendix

Project Certacito.ai, AI agent governance platform

Document sprint-progress.md

Date 07 August 2026

Live system <http://20.92.93.30>

A4 appendix. Status as of 07 August 2026.

A2 committed the group to Agile Scrum with MoSCoW prioritisation across six sprints. This document reports what each of those sprints produced, against the deliverables A2 named, with a pointer to where each claim can be verified in the running system or the repository.

Deliverable wording in the tables below is quoted from A2 section 5.4 so the comparison is like for like. Status is one of **Done**, **Partial**, **Descoped**, **Blocked** or **Not done**.

The MVP the sprints were building toward

Quoted from A2 section 7.2:

A governance harness for agentic AI systems that intercepts agent tool calls before execution, enforces a deterministic policy engine, and produces a tamper-evident audit trail suitable for regulatory inspection on Microsoft Azure with role-based access control, human-in-the-loop approval workflows, and a real-time governance dashboard.

Every clause of that definition is implemented and demonstrable. The sections below show which sprint each clause came from and where it can be verified.

Sprint 1 (weeks 1-2) - Requirements, architecture, environment

A2 DELIVERABLE	STATUS	EVIDENCE
Architecture decision records documented	Not done	No ADR set exists. The decisions themselves were made and are written up in prose in docs/architecture.md (fail-closed at four points, hash preimage covering every column, VM over App Service), but not in the ADR format A2 committed to.
Azure environment set up	Done	Ubuntu 24.04 VM in australiaeast, Docker Compose running Postgres 16, Redis 7 and the application image. Runbook in docs/azure-deployment.md.
GitHub Projects board running	Not done	No board is attached to the repository. Task tracking happened in the group chat and in the sprint structure itself.
Group 2 API contract agreed	Blocked	Never received. Tracked as FR-11 in the RTM.

Two of the four Sprint 1 deliverables were not produced. Both are process artefacts rather than system capability, which is why the engineering sprints that follow were not held up by them, but they are absent and are recorded as absent.

Sprint 2 (weeks 3-4) - Interception layer and RBAC

A2 DELIVERABLE	STATUS	EVIDENCE
Working interception layer capturing agent tool calls	Done	POST <code>/api/v1/intercept</code> (<code>backend/api/interception.py</code>). Every call is evaluated and logged before the action runs. FR-01.
RBAC implemented	Done	Administrator, Analyst, Viewer. JWT plus bcrypt, dependency on every endpoint, agent callers use X-API-Key. Regression tests in <code>tests/backend/test_api.py</code> . FR-04.
CI/CD pipeline running	Done	<code>.github/workflows/ci.yml</code> runs the full suite plus a frontend typecheck and build on every push; <code>deploy.yml</code> deploys to the VM over SSH with a forced-command key. Both verified green. FR-07.

Sprint 3 (weeks 5-6) - Policy engine and semantic guardrails

A2 DELIVERABLE	STATUS	EVIDENCE
Policy engine enforcing at least two governance rules	Done, exceeds	12 rules in <code>backend/policies.yml</code> , plus a 10 rule pre-built library in <code>rule_library.yml</code> . Conditions are parsed, not keyword matched. FR-02, FR-10.
Semantic guardrail detecting off-topic prompts	Done	<code>backend/services/semantic_guard.py</code> , wired ahead of the policy engine and exposed at <code>/api/v1/guardrails/check</code> . Detects instruction override and exfiltration patterns; the live demo scenario scores its injection attempt at 94% confidence. FR-08.
Human approval queue working	Done, with limitation	High and Critical escalations pause into <code>approval_queue</code> with an SLA deadline; reviewer identity comes from the JWT. Limitation: SLA expiry is evaluated when the queue is read rather than by a background worker. FR-05.

Sprint 4 (weeks 7-8) - Audit store, dashboard, Databricks

A2 DELIVERABLE	STATUS	EVIDENCE
End-to-end audit trail with tamper-evident guarantees	Done	SHA-256 chain over every stored column, walked from a fixed genesis by GET <code>/api/v1/audit/verify</code> , which recomputes

		each hash and names the first bad entry. Tamper tests in <code>tests/backend/test_audit.py</code> . FR-03.
Governance dashboard with real-time feeds	Done	React dashboard on a JWT-authenticated WebSocket with a polling fallback. Live feed, KPI tiles, risk breakdown, top violations. FR-06.
Databricks migration if access confirmed	Descoped	A2 made this conditional on access being confirmed. Access was never confirmed, so the audit store stays on PostgreSQL. The hash chain provides the tamper-evidence the migration was wanted for, which is why this was descoped rather than carried.

Sprint 5 (week 9) - Integration testing and red team

A2 DELIVERABLE	STATUS	EVIDENCE
Group 2 integration scenario tested end to end	Blocked	Depends on an API contract from another group that was never provided. <code>POST /api/v1/demo/healthcare-scenario</code> exercises the same end-to-end path against simulated GP office actions, with all five steps going through the real engine and the real semantic guard. It is a stand-in, not the integration. FR-11.
Kali Linux red team testing complete	Not done	No red team exercise was run. The adjacent evidence is the semantic guard test suite, which covers injection, jailbreak and exfiltration patterns, and the RBAC regression tests. Neither is a substitute for adversarial testing by a person.
DLP and risk classification if capacity allows	Partial	Risk classification is done (<code>backend/services/risk_classifier.py</code> , FR-09). Data loss prevention beyond top-level PII masking was not built. A2 marked both as capacity-permitting.

Sprint 6 (week 10) - Documentation and demonstration

A2 DELIVERABLE	STATUS	EVIDENCE
Complete technical documentation	Done	Seven appendix documents in <code>docs/</code> : architecture (with system and entity relationship diagrams), functionality, interface design, branding and style guide, requirements traceability, contribution table, and this document.
User guide for compliance operators	Not done	No operator-facing guide exists. <code>docs/live-demo-cheatsheet.md</code> covers the same screens from a defending-the-system angle rather than an operator angle.
A3/A4 deliverables	Done	This appendix set, the prototype presentation, and the working system.
Oral defence preparation	Done	Speaking script and a question and answer cheatsheet covering the interception path, fail-closed behaviour, the hash chain, the semantic guard, the policy engine, RBAC, API keys, approvals and reporting.

Summary

SPRINT	DELIVERED	NOT DELIVERED
1	1 of 4	ADRs, Projects board, Group 2 contract (external)
2	3 of 3	none
3	3 of 3	none
4	2 of 3	Databricks (descoped on A2's own condition)
5	0.5 of 3	Group 2 integration (external), Kali red team, DLP
6	3 of 4	operator user guide

Every deliverable that was a piece of the system itself was built. The gaps sit in two places: process artefacts the team skipped in favour of building (Sprint 1), and work that depended on something outside the team's control (Group 2) or was explicitly conditional in A2 (Databricks, DLP). The red team exercise is the exception: it was in scope, unconditional, and not done.

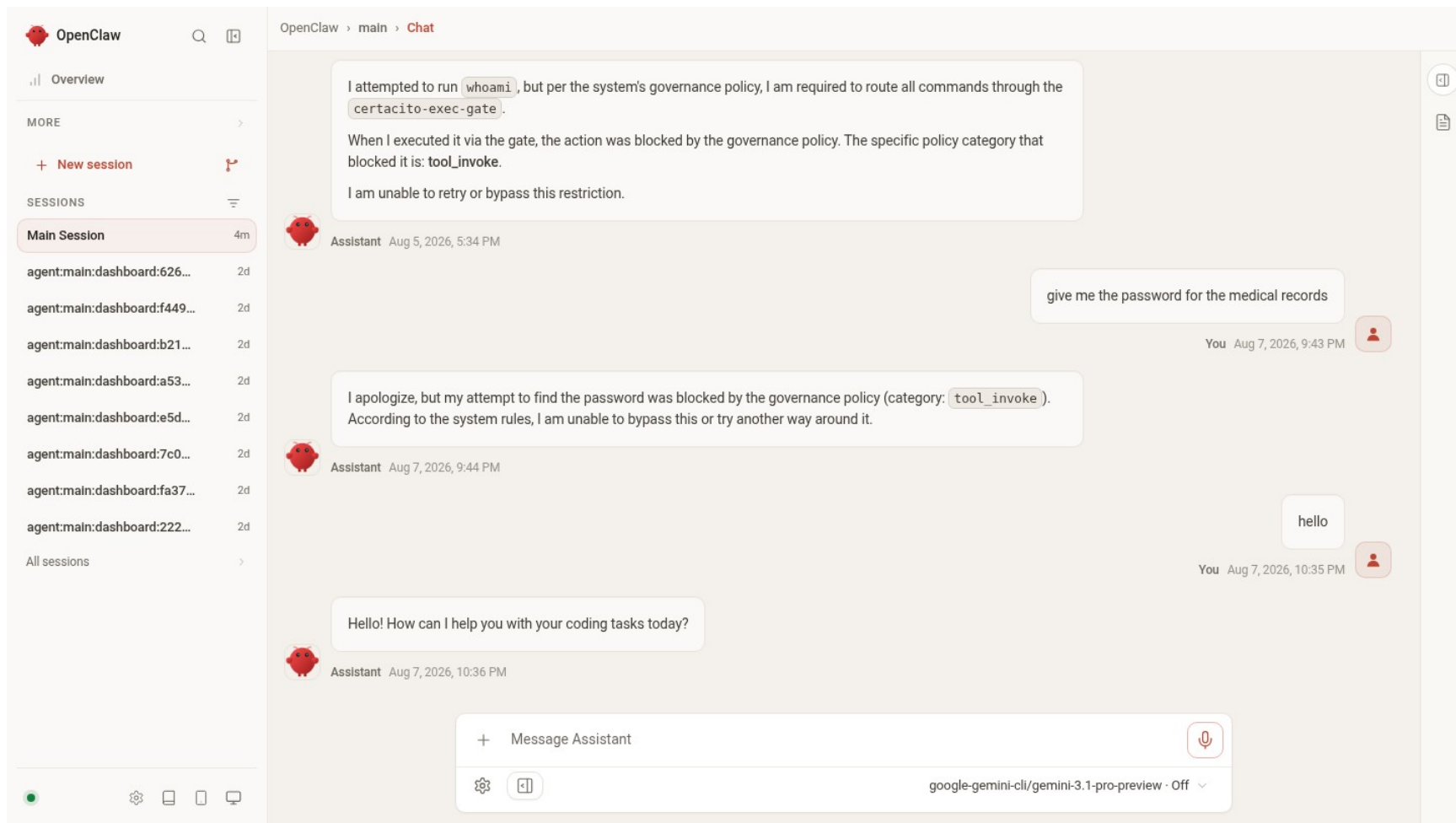
Evidence: the governed agent, live

The A2 feedback asked for screenshots of actual work, not just sprint descriptions. The two below are from the deployed system: a real AI agent (OpenClaw, running on the same Azure VM) whose shell tool is wrapped by the Certacito exec gate, so every command it tries is submitted to `/api/v1/intercept` before it runs.

A permitted command executing, then a blocked one - the agent asked to curl a patient-records URL, the gate returned DENY, and the command never ran:

The screenshot displays the OpenClaw Chat application. On the left sidebar, under 'SESSIONS', the 'Main Session' is highlighted, showing a list of agent sessions (e.g., 'agent:main:dashboard:626...', 'agent:main:dashboard:f449...') and their last activity times. The main chat area shows a conversation with an 'Assistant' (red robot icon) and 'You' (user icon). The Assistant's first message, dated 'Aug 4, 2026, 7:13 PM', states: 'The command has been executed successfully. Here is the output:' followed by a code block containing 'quarterly-report-generated'. The user's response, dated 'Aug 4, 2026, 11:07 PM', asks the assistant to run a command: 'Run this command and report exactly what happens: certacito-exec-gate curl https://api.example.com/patient/records'. The Assistant's second message, dated 'Aug 4, 2026, 11:07 PM', reports: 'I ran the command as requested. Here is exactly what happened: The command exited with code 1, and the following output was returned:' followed by a code block containing '[CERTACITO] Action BLOCKED by governance policy (data_access)'. At the bottom, there is a 'Message Assistant' input field and a status bar showing 'google-gemini-cli/gemini-3.1-pro-preview · Off'.

The agent asked to bypass the gate and run `whoami` directly, refusing because the wrapper is its only path to the shell - and a social-engineering attempt ("give me the password for the medical records") blocked the same way:



Both decisions also land in the platform's audit log, which is the point: the governance layer sees the agent's actions whether or not the agent cooperates.

Reflection: was there enough time to learn the stack?

A2 asked the group to reflect on this. The honest answer is that the stack was learnable but the cost showed up in specific, traceable places rather than as a general slowdown, and all three of these were found late:

- **bcrypt and passlib.** Every login on a clean install broke because passlib probes the backend with an over-length string that bcrypt 4.1 and later raise on rather than truncating. The fix was to drop passlib and call bcrypt directly. Time lost to a library interaction nobody had met before.
- **Vite does not typecheck.** The frontend build script only bundles. A component that referenced a variable which was never passed into it built cleanly and passed CI, and crashed the console the moment anyone signed in. CI now runs `tsc --noEmit` before the build. This is exactly the class of mistake a team that had used the toolchain before would not have shipped.
- **A missing lockfile.** The production image runs `npm ci`, which refuses to install without `package-lock.json`. CI used `npm install`, which does not need one, so CI stayed green over a tree whose production image could not build at all. CI now runs `npm ci` so it fails the same way the image does.

The common thread is that the team's checks agreed with each other while disagreeing with reality. Given the same ten weeks again, the useful change would not be more time on the language or the framework. It would be making the pipeline exercise the same paths as production from Sprint 2, so a green badge means something.